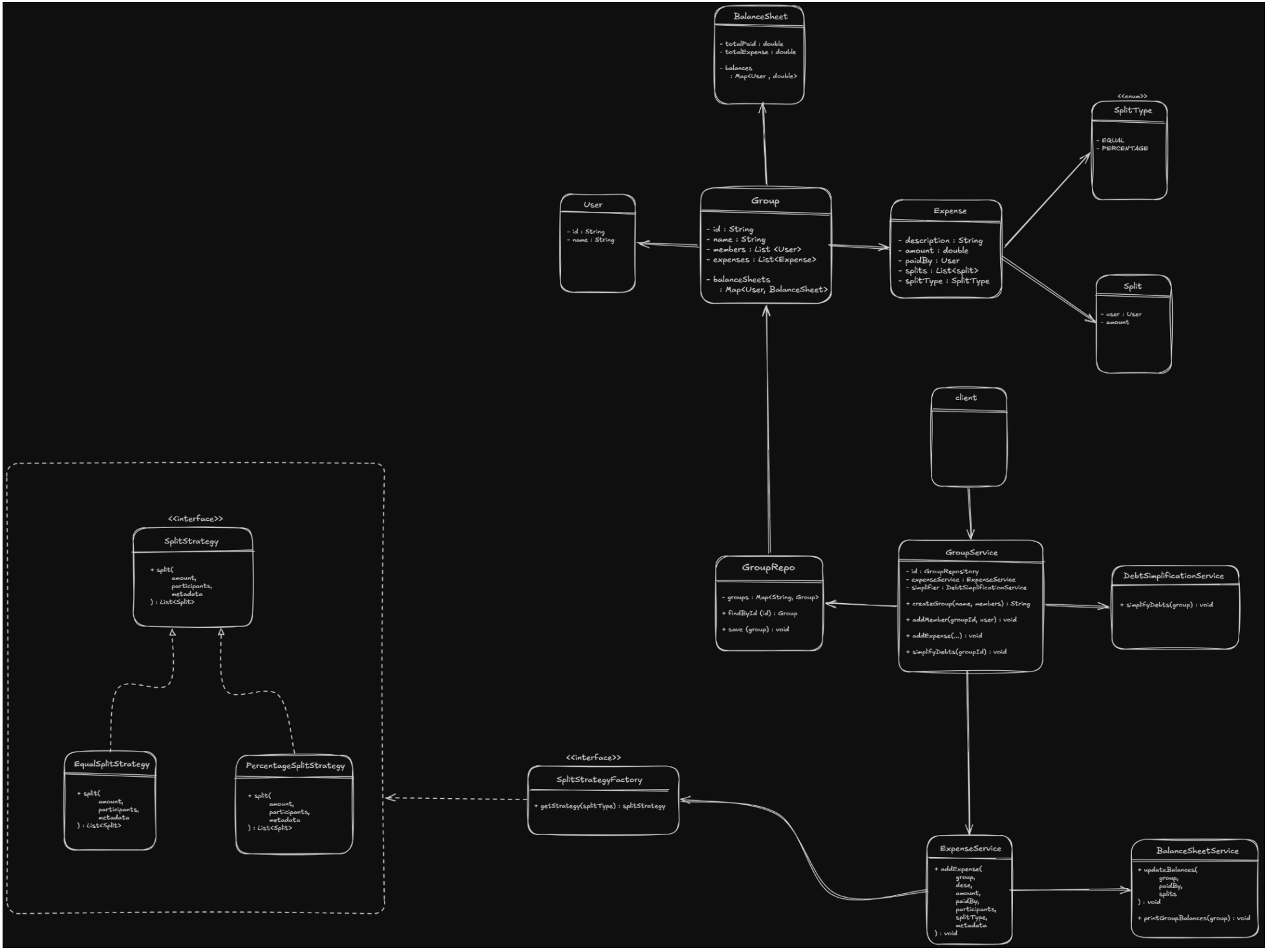
BalanceSheet
- totalPaid : double
- totalExpense : double
- balances
: Map<User , double>
<<enum>>
SplitType
- EQUAL
- PERCENTAGE
User
- id : String
- name : String
Group
- id : String
- name : String
- members : List <User>
- expenses : List<Expense>
- balanceSheets
: Map<User, BalanceSheet>
Expense
- description : String
- amount : double
- paidBy : User
- splits : List<split>
- splitType : SplitType
Split
- user : User
- amount
client
<<interface>>
SplitStrategy
+ split(
amount,
participants,
metadata
) : List<Split>
GroupRepo
- groups : Map<String, Group>
+ findById (id) : Group
+ save (group) : void
GroupService
- id : GroupRepository
- expenseService : ExpenseService
- simplifier : DebtSimplificationService
+ createGroup(name, members) : String
+ addMember(groupId, user) : void
+ addExpense(...) : void
+ simplifyDebts(groupId) : void
DebtSimplificationService
+ simplifyDebts(group) : void
EqualSplitStrategy
+ split(
amount,
participants,
metadata
) : List<Split>
PercentageSplitStrategy
+ split(
amount,
participants,
metadata
) : List<Split>
<<interface>>
SplitStrategyFactory
+ getStrategy(splitType) : splitStrategy
ExpenseService
+ addExpense(
group,
desc,
amount,
paidBy,
participants,
splitType,
metadata
) : void
BalanceSheetService
+ updateBalances(
group,
paidBy,
splits
) : void
+ printGroupBalances(group) : void